WitnessKey / LOOPtLOOP Vision Note

Living Consent Loops, Witness Mandates, and the Path Beyond Static Receipts

Prepared for continuation after the first working Witness prototype. 2026-06-20.

> **Tomorrow-morning reminder: you are not merely building a prettier click-through. You now have a functioning prototype of a real witness. The next insight is that the receipt is not the permission. The receipt is the handle to a living permission state.**

1. What exists now

The current prototype proves that a private authorization can be hashed locally, witnessed by a third party without disclosing the private payload, accepted through Abracadoo, recorded by the LOOPtLOOP API, signed as a receipt, and verified through WitnessKey.

```
WitnessMark creates offer
-> Abracadoo accepts witness loop
-> LOOPtLOOP records and signs event
-> WitnessKey verifies receipt
-> Private payload remains local
```

This is already useful. It is a working hash-witnessed authorization receipt system. But the deeper substrate becomes visible only when the receipt is treated as the birth certificate of a living loop, not as the loop itself.

2. The key distinction

A static receipt answers: Was consent witnessed at time T0?

A living consent loop answers: Was consent witnessed at time T0, and is it still valid at time T1 for the action now being contemplated?

This is the difference between platform-local click confirmation and revocable consent-chain infrastructure.

3. The full loop

1. A private agreement or authorization is created between Party A and Party B.
2. The agreement cleartext may remain private to the parties, their local tools, or their chosen vaults.
3. The cleartext is hashed locally.
4. A and B consent to use Witness W for this loop, either explicitly in this instance or through a prior acceptable-witness relationship.
5. Witness W consents per instance to perform a bounded witness function.
6. W receives the hash, consent envelope, role declarations, timestamps, schema/code version, and witness scope - not the private context unless expressly included.
7. W signs the receipt and maintains the live status surface.

8. Either authorized party can later revoke, expire, supersede, or challenge the loop according to its rules.
9. A downstream actor can check the live status before relying on the authorization.
10. If cleartext is later revealed locally, it can be hashed and compared to the witnessed hash without asking W to possess or reconstruct it.

4. Witness Mandate: W is a participant, not a passive endpoint

The witness must consent to the witness role. This is what makes W accountable without making W omniscient. W does not own the private context; W owns the accountability of its witness function.

- A and B consent to use W as an acceptable witness for the loop.
- W consents to witness this instance or class of instances.
- W declares what it will receive, store, sign, verify, revoke, and expose.
- W declares what it does not claim: truth, authorship, legal identity, legal capacity, absence of coercion, or completion of downstream action.
- W can be held accountable for the bounded function it actually performs: hash receipt, signature, status maintenance, revocation handling, and honest verification responses.

> **Canonical formulation: The witness does not own the context; the witness owns the accountability of its witness function.**

5. Revocation is not an add-on; it is what makes the loop alive

A consent artifact that cannot be revoked becomes a trap. A living consent loop must expose a reachable status surface so that consequential actors can ask whether the loop still stands.

- The original receipt remains historically true: a consent loop was born.
- The current status may change: active, revoked, expired, superseded, invalidated, or challenged.
- Either side may withdraw its role-appropriate consent.
- W must record and expose the current status within the agreed visibility bounds.
- A relying actor should check status before consequential action, not merely rely on an old receipt.

```
Receipt = origin witness
Status = current life of the loop
Reliance check = action-time accountability
```

6. Downstream reliance without contextual exposure

The next-order value is that downstream actors can act on live consent-chain references without knowing the private context of the original agreement. They can rely on assertions from one party while still checking whether the underlying consent loop is currently active.

- Private context stays private.
- Reliance stays accountable.
- Consent stays revocable.

A downstream actor does not need the full agreement text. It needs a bounded assertion, a witness reference, a live status check, and a record that it checked before acting.

```
consent -> witness -> assertion -> reliance check -> action
```

This prevents two failure modes: over-disclosure, where every downstream actor demands the private context; and blind trust, where actors rely on stale or unverifiable assertions. The living loop creates a third option: revocable consent-chain reliance.

7. The semantic horizon

For now, exact hashes are the right implementation substrate. They are simple, auditable, and already working. The semantic layer should not be rushed into the prototype.

But the long horizon is larger: agreements may eventually be made not only over exact syntax, but over semantic attractors - bounded regions of meaning that can survive translation across language, modality, and agent interpretation, provided the interpretation protocol and capacity checks are themselves witnessed.

This does not require WitnessKey to invent the whole semantic protocol. It can leverage future external work - possibly Cisco, open source initiatives, or other efforts toward a Semantic State Transfer Protocol or equivalent semantic interoperability layer. LOOPtLOOP should leave schema room for this without pretending to solve it now.

- Today: exact_text_hash or structured_json_hash.
- Soon: local payload match and revocation/status.
- Later: semantic_attractor_ref, translation protocol, capacity checks, and semantic equivalence witness claims.

The goal is not to replace text prematurely. The goal is to avoid building a system that can only ever understand consent as bytes. Humans consent to meanings, roles, boundaries, affordances, and consequences. Exact hashes are the first stable bridge.

8. What not to forget

- This is not just a receipt service. It is the beginning of living authorization infrastructure.
- The prototype is already real: WitnessKey/LOOPtLOOP can witness a private authorization hash and verify the signed receipt.
- The missing next product clarity is local payload match plus live revocation/status.
- The witness must be a consenting, accountable role participant, not a magical oracle.
- Downstream actors should be able to cite and monitor consent-chain references without extracting private context.
- The semantic future is real, but it belongs after exact-hash loops, status, revocation, and reliance checks are solid.

9. Near-term roadmap

v0.1 - Birth, already functioning

- Create private authorization witness offer.
- Accept through Abracadoo.
- Record signed receipt through LOOPtLOOP API.
- Verify receipt through WitnessKey.

v0.2 - Match

- Add local private payload match on the WitnessKey verifier.
- User pastes cleartext locally; browser hashes it; no upload occurs.
- Verifier shows whether local cleartext matches the witnessed payload hash.

v0.3 - Life

- Add revoke/status endpoints.
- Expose current authorization state: active, revoked, expired, superseded, invalidated, challenged.
- Record who revoked or withdrew role consent, within privacy bounds.

v0.4 - Reliance

- Add reliance-check endpoint for downstream actors.
- Record status-at-action-time without revealing private context.
- Let an AI agent, app, or service cite a consent chain before consequential action.

v0.5 - Witness Mandates

- Make witness acceptance explicit.
- Record W consent-to-witness, W scope, W non-scope, and W accountability claims.

Future - Semantic Attractors

- Add schema slots for semantic agreement objects once external semantic transfer protocols mature.
- Support translation/capacity-aware meaning commitments without pretending that v0.1 hashes solve semantics.

10. One-paragraph vision

> **WitnessKey and LOOPtLOOP are evolving from hash-witnessed authorization into living consent-chain infrastructure. Parties consent not only to an agreement but to the witness that will maintain its revocation and status surface; the witness consents to a bounded, accountable role; downstream actors can rely on live consent-chain references without accessing private context; and, eventually, agreement objects can move beyond exact syntax into tested semantic attractors that preserve meaning across language, modality, and agent interpretation. The current prototype is the first working witness: small, exact-hash based, and real.**

11. Mantra for tomorrow

> **The receipt witnesses origin; the status endpoint witnesses life.**
>
> **The private context can remain veiled; the consent state must remain reachable.**
>
> **A downstream actor should not extract context when a live consent-chain reference will do.**
>
> **The witness does not own the context; the witness owns the accountability of its witness function.**
>
> **Private context stays private. Reliance stays accountable. Consent stays revocable.**